SKIM — Personal AI Project Blueprint

Version: 1.0
Goal: Zero-budget, local-first, voice-first personal AI
Development environment: Google Antigravity
Initial hardware: Apple Silicon Mac with 16 GB RAM

1. Product Vision

SKIM is a personal AI assistant designed to capture spoken updates, maintain a structured history of daily life, research topics, and analyze progress over time.

Its four core capabilities are:

- **Capture:** Record voice and transcribe it.
- **Remember:** Store personal facts, activities, decisions, goals, and documents persistently.
- **Research:** Answer questions using personal knowledge and optional external sources.
- **Analyze:** Calculate trends, summarize progress, and identify possible patterns.

SKIM is not primarily a generic chatbot or a creative-design tool.

2. Recommended Architecture

1. **Voice interface:** Record, transcribe, review, and speak responses.
2. **Processing pipeline:** Speech-to-text → intent detection → structured extraction → validation.
3. **Personal memory:** Raw transcripts, dated daily logs, structured records, and source links.
4. **Knowledge retrieval:** Search documents and personal notes; add semantic search when useful.
5. **Local AI:** Ollama model for language understanding, summarization, and extraction.
6. **Analytics engine:** Python calculations and reports based on stored records.
7. **Dashboard:** Daily timeline, search, charts, research, and reports.

3. Technology Stack

Layer	Technology	Purpose
IDE	Google Antigravity	AI-assisted development
Language	Python	Orchestration, data processing, analytics
Backend	FastAPI	Local API and application logic
Frontend	React + Vite + TypeScript	Desktop-friendly interface
Local AI	Ollama	Run language models locally
Speech recognition	Faster-Whisper / whisper.cpp	Local voice transcription (Apple Silicon)
Database	SQLite	Persistent personal records
Analytics	pandas	Statistics, trends, and deterministic reports

Layer	Technology	Purpose
Document search	SQLite FTS5	Search notes, protocols, and documents
Charts	Recharts	Visualize progress & metrics dynamically

Budget principle: Use open-source software and local models wherever practical. Optional external research services may have limits or paid tiers, so verify their terms before depending on them.

4. Functional Modules

Module 1 — Voice Capture

- Push-to-talk recording, local transcription, editable transcript, timestamps, and optional spoken responses.
- Save the original transcript before extracting structured information.

Module 2 — Personal Memory

- Store daily logs, facts, preferences, decisions, goals, and documents.
- Support search, corrections, export, and deletion.
- Keep original statements separate from AI-generated summaries.

Module 3 — Life Trackers

- Separate records for fitness, health measurements, sleep, nutrition, study, work, finance, habits, and goals.
- Every record should have a date, category, units where relevant, and optional notes.

Module 4 — Research Assistant

- Search personal documents and optionally the web.
- Retain source URLs and retrieval dates.
- Separate verified findings from hypotheses and unsupported claims.

Module 5 — Analysis and Reports

- Calculate weekly and monthly statistics.
- Compare progress against goals and generate charts.
- Ground insights in stored data; do not invent missing records.

Module 6 — Privacy and Reliability

- Local storage, backups, validation, visible corrections, and configurable retention.
- Require approval before destructive actions or external sharing.

5. Initial Database Design

Table	Purpose
voice_sessions	Recording metadata and transcription status
transcripts	Original transcript, timestamp, and corrections
daily_logs	Daily summaries linked to source transcripts
activities	Gym, study, work, and other activities

Table	Purpose
measurements	Weight and other numeric measurements with units
goals	Target values, deadlines, and status
documents	Imported files and their metadata
research_sources	Source URLs, titles, and retrieval dates
insights	Generated analyses linked to underlying records

Every extracted record should link back to its source transcript. Add a review status for uncertain extractions. Never overwrite a confirmed measurement simply because a later AI response suggests a different value.

6. Development Roadmap

Phase 1 — Foundation and Local AI

Create the repository, Python environment, application structure, and Ollama connection. Test a local model.

Completion test: A local prompt works.

Phase 2 — Voice

Add recording controls, local transcription, transcript editing, and save/replay.

Completion test: Voice becomes saved text.

Phase 3 — Memory

Add SQLite tables, extraction, validation, corrections, and historical retrieval.

Completion test: Restart the app and retrieve an old entry.

Phase 4 — First Tracker

Implement weight and gym tracking, calculations, charts, and a weekly report.

Completion test: Report values match the database.

Phase 5 — Research

Add document import, personal knowledge search, optional web retrieval, citations, and source-backed answers.

Phase 6 — Expansion

Add study, work, finance, goals, reminders, and approved task execution.

Do not build all phases at once. Each phase must work and be tested before moving to the next.

7. Antigravity Implementation Brief

Create a folder named **`skim`** on your Mac, open it as a project in Google Antigravity, and give the agent the following instructions. The first implementation should be limited to Phase 1.

```
# SKIM — Project Instructions

## Mission
Build a zero-budget, local-first, voice-first personal AI assistant for one
```

```
user. SKIM captures spoken daily updates, transcribes them, stores
structured personal records, searches personal knowledge, performs research,
and analyzes progress over time. The user is a beginner; explain commands
and installation steps clearly.

## Non-negotiable requirements
1. No paid APIs, subscriptions, or cloud AI dependency in the core workflow.
2. Use Ollama for local language-model inference.
3. Use local speech recognition, initially evaluating whisper.cpp.
4. Store persistent data locally in SQLite.
5. Preserve original transcripts and link extracted records to their source.
6. Separate facts, goals, estimates, and AI-generated interpretations.
7. Use deterministic Python code for numerical calculations.
8. Require confirmation for uncertain extractions and destructive actions.
9. Provide data export, correction, deletion, and backup mechanisms.
10. No creative-design generation module is required.
11. External research is optional and must retain source URLs and retrieval
dates.
12. Do not upload personal records to external services without explicit
user consent.

## Preferred architecture
- Backend: Python and FastAPI.
- Frontend: React, Vite, TypeScript.
- Database: SQLite.
- Local LLM: Ollama.
- Speech recognition: whisper.cpp or another verified open-source local
option.
- Analytics: Python and pandas.
- Tests: pytest for backend logic and appropriate frontend tests.

Keep frontend and backend separate. Use environment configuration for local
service addresses. Never commit secrets, private data, recordings, or
database files.

## Initial scope — Phase 1 only
- Inspect the existing environment.
- Document prerequisites and current installation status.
- Create the initial project structure.
- Implement a health endpoint.
- Implement a configurable Ollama connection.
- Add one integration test for local model connectivity and unit tests for
configuration handling.
- Write a README with beginner-friendly setup instructions.
- Avoid installing unnecessary dependencies.

Do not start microphone capture, expanded database schemas, research tools,
or autonomous computer control until Phase 1 is reviewed and approved.

## Engineering rules
- Inspect the workspace before editing.
- Propose a short implementation plan first.
- Ask before installing system-level software or running destructive
```

```
commands.
- Do not overwrite user files without approval.
- Test every implemented feature.
- Report what changed, what passed, what failed, and the next recommended
step.
- Never claim a feature works unless it has been tested.

Begin by inspecting the workspace and presenting the Phase 1 plan. Do not
implement the entire application in one pass.
```

8. Risks and Constraints

- A 16 GB Apple Silicon Mac is a reasonable starting point for small local models, but speed and memory usage vary.
- Local processing remains private only if the application is configured not to transmit data elsewhere.
- The assistant cannot know activities that were not recorded or provided through connected data sources.
- AI-generated analysis must be grounded in stored data; correlation is not proof of causation.
- The IDE may use online AI services. Keep sensitive personal records out of prompts and external agent context unless sharing is explicitly approved.
- Open-source does not automatically mean every optional service, model, or feature is free; verify license and usage terms.

9. Official Starting Resources

- Google Antigravity: https://antigravity.google/docs/getting-started
- Ollama: https://ollama.com/
- whisper.cpp: https://github.com/ggml-org/whisper.cpp
- FastAPI documentation: https://fastapi.tiangolo.com/
- React documentation: https://react.dev/

First Milestone

The first milestone is not a complete personal AI. It is a small, working local application that connects to Ollama and passes a test. Once that foundation is stable, voice and persistent memory can be implemented as separate, testable engineering tasks.